

Integrating and Testing Zero-Knowledge Presentations in Swiyu

Antonio Kambiré Martín Ochoa
zkSecurity

Abstract

Swiyu’s digital credentials use selective disclosure, while zero-knowledge proofs can establish predicates over those credentials without revealing the underlying attributes. Integrating different proof systems requires a common presentation flow and an explicit account of what each circuit proves. We built a provider interface and an experimental extension to Swiyu’s OpenID for Verifiable Presentations flow, with stage measurements and automated transcript tests. We also authored an OpenAC-based circuit whose declared statement matches the eid-privacy age-25 circuit, registered both against one semantic claim, and implemented comparisons within and across providers. The tests ask whether presentations with the same permitted meaning produce equivalent observer transcripts. Native provider campaigns and deterministic cross-provider regressions exercise the workflow; injected credential disclosures and correlation tags are detected even when verification succeeds. The framework gives contributors one integration path for their circuits, benchmarks, and privacy tests.

1 Introduction

Swiyu is Switzerland’s trust infrastructure for digital credentials. An issuer signs a credential and delivers it to the holder’s wallet; the holder later presents selected information to a verifier. The system includes issuance and verification services, a wallet, and registries for public keys and trust information. It uses OpenID for Verifiable Credential Issuance (OID4VCI) for issuance and OpenID for Verifiable Presentations (OID4VP) for presentation [11].

Swiyu credentials use SD-JWT selective disclosure. The issuer signs a payload containing salted commitments to disclosable attributes. The wallet sends that payload and the requested disclosures, and demonstrates control of its holder key [6, 11]. This lets a verifier request a birth date without receiving a name or address, but the verifier still learns the requested date. Stable signed material and holder keys can also link repeated presentations [6]. Swiyu’s batch-issuance design addresses credential reuse by issuing credentials with separate holder keys and renewing them [2]. Predicates over hidden attributes require a further mechanism.

A zero-knowledge proof can establish that an authen-

ticated credential satisfies an age threshold while keeping its birth date private [8, 14]. The integration must also preserve that privacy: a proof may hide the credential while an envelope field or HTTP header exposes it. Different proof systems bring different circuits, public inputs, preparation steps, and proof encodings. Evaluating them therefore requires both an agreed statement and a shared protocol path.

We built an integration and testing layer for this purpose. It connects proof providers to Swiyu’s presentation flow, measures their execution stages, and compares captured exchanges under a declared disclosure policy. We made the comparison concrete by writing an OpenAC-based circuit aligned with eid-privacy’s age-25 statement. The shared claim identifies which executions should reveal the same information; differential tests check whether the surrounding integration preserves that relation.

2 From credentials to ZK presentations

OpenAC/zkID provides a route from signed credentials to anonymous presentations. Its Prepare/Show design separates credential preparation from subsequent presentations [14]. We used its circuit and adapter stack to implement Swiyu-specific profiles: an age threshold with credential validity and private status checks, canton allow-list membership, residence eligibility, and a scoped one-use claim. These profiles exercise issuer authentication, holder authorization, attribute extraction, and verifier-chosen policy inputs in one presentation [9].

The eid-privacy project supplies another implementation path through Noir circuits, including credential and holder authentication and age checks [5]. Its d10 circuit is the basis of our second provider, labelled EPFL in the implementation. We run it with UltraHonk. The project’s separate Spartan backend illustrates that circuit language, proof backend, and presentation adapter are distinct choices [4]. Each contributes different artifacts and execution stages.

A common proof API is only part of the integration. The verifier must request the appropriate statement, the wallet must bind its response to that request, and both sides must agree on the public inputs. An age-and-status circuit and an age-only circuit also answer different questions. We therefore separate the protocol adapter, the

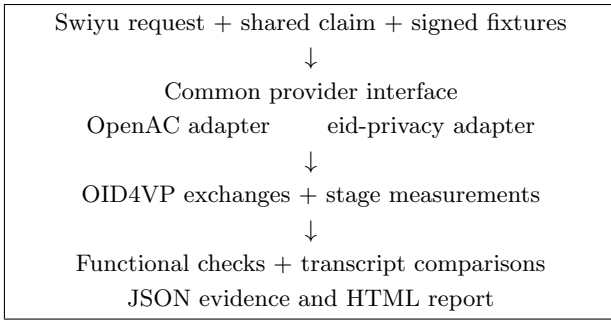


Figure 1: The integration layer connects provider implementations to the same presentation flow and evaluation stages. Claims determine which executions can be compared.

semantic claim, and the provider’s implementation of that claim. This lets a new backend reuse the presentation and testing machinery while declaring its own supported statements.

3 Generic integration layer

3.1 Connecting a provider

Our extension to the Swiyu Generic Verifier carries an experimental `x.swiyu.zkp` policy in the signed DCQL request. The wallet returns an opaque proof envelope in `vp.token` through `direct.post`. The verifier routes the selected profile to its provider and checks the expected session context. This uses the existing OID4VP request and response path [13, 12, 9].

Providers expose `initialize`, `prepare`, `present`, `verify`, and `cleanup` through a local process contract. They own credential conversion, circuit artifacts, keys, witness construction, and cryptographic operations. Shared code handles lifecycle calls, request binding, transport capture, and reports. A manifest declares the executable and supported profiles; a support file binds those profiles to versioned claims and artifact hashes. Contributor declarations are checked against the shared claim catalog before execution [9].

The runner records stage durations, proof and witness sizes when available, functional outcomes, and transcript findings in JSON and HTML. A contributor supplies the adapter, artifacts, support declaration, and fixture conversion, then runs the same local checks as other providers. Cryptographic audits remain separate from this executable integration contract.

3.2 Bounded claim contract

A *claim* describes the complete integration acceptance relation and its permitted releases. It names one credential source, authenticated attribute paths and types, verifier-supplied inputs, mandatory assertions, and a bounded `all/any` predicate expression. An age predicate refers to a birth-date attribute extracted from the

authenticated credential. General composition belongs to the shared claim model; a provider declares the subset it implements.

Mandatory assertions cover such obligations as issuer authentication, expected metadata, credential validity, holder authorization, request binding, and status at a supplied reference. A particular claim selects its obligations explicitly. Attribute predicates compose below these assertions: an OR between two business predicates cannot bypass issuer authentication. A new status-free claim has a different identity from one requiring status checks. The contract also declares permitted semantic releases and opaque cryptographic channels, following the information-flow distinction between confidential data and authorized declassification [10].

The shared age-25 claim requires issuer authentication under a given key, an authenticated birth-date disclosure, age of at least 25 at a given reference date, and holder authorization over a given challenge. It permits acceptance and an opaque proof. Here, “given issuer” means that the verifier supplies the expected key. Issuer trust, the reference date, and the challenge belong to the public context supplied by the experiment. Claims with additional validity or status obligations can use the same model under their own identities [9].

Versioned catalog entries fix argument types, attribute extraction, assertion meanings, and fixture evaluation. A canonical statement digest identifies the claim. A separate implementation declaration binds that digest to provider artifacts, supported input domains, and the claimed placement of checks in the proof or verifier. Campaign identity records the concrete fixtures and execution configuration. The declaration records the statement claimed by an implementation; circuit correspondence remains an audit obligation [9].

3.3 Aligning the circuit statements

We implemented `swiyu.age25.jwt` in Circom over the OpenAC adapter to match the statement of the eid-privacy `d10` Noir circuit (locally labelled EPFL) [14, 5]. The starting OpenAC circuit combined an age-18 cutoff with validity, metadata, status, and session-context checks. The new circuit retains issuer ES256 verification and authentication of the birth-date disclosure, replaces the cutoff with `nowDate ≥ birthDate + 25 · 10000` on `YYYYMMDD` integers, and checks a holder P-256 signature over the public challenge. Validity, metadata, and status checks belong to the original profile; the shared profile selects the four obligations above. The adapter derives the public challenge from the verifier session and policy [9].

Both support declarations register `swiyu.shared.age25-holder-challenge.v0` and the same statement digest, while retaining their own circuit and profile identifiers. A common conformance corpus checks two adult credentials, the exact age

boundary, an underage credential, a wrong issuer key, a corrupted holder signature, and a changed challenge. The corpus checks the reference evaluator and the two registrations against the shared contract [9].

4 Differential leakage testing

4.1 Assembling the transcript

The experiments capture four application HTTP exchanges: management creation of a verification request, retrieval of its signed request object, wallet `direct_post`, and retrieval of the management result. Each capture includes method, URL, headers, status, body, and order. This is a verifier-side observation model augmented with management traces; an arbitrary network observer would not necessarily see the same plaintext or endpoints [9].

The capture covers these four application boundaries. We decode their transport bodies and retain unknown fields alongside known protocol fields, so that additional data contributed by an adapter remains visible.

4.2 Choosing comparable presentations

Let C identify a claim, w a private credential fixture, g the public verifier inputs, and $R_C(w, g)$ the permitted release. The intended relation compares executions whose credentials authenticate, whose expected acceptance agrees, and whose releases and public inputs agree modulo declared session freshness. For eligible executions i, j , the tested condition is

$$N_P(O(T_i)) = N_P(O(T_j)), \quad (1)$$

where T is a captured execution, O selects the observer’s boundaries, and N_P applies the declared normalization policy P . Eligibility is checked before this equality is interpreted as a privacy test.

The independent classifier verifies SD-JWT issuer signatures and the supported metadata, validity, and attribute predicates. The submitted verifier evaluates holder and native status obligations, and the report records this division of checks. An eligible privacy pair requires the verifier to agree with the expected acceptance [9].

Each campaign creates fresh verifier sessions. Paired fixtures change a hidden input while keeping the claim and allowed release fixed. A same-credential repeat records one reference for session-induced variation before credentials are changed. Incorrect signatures and mismatched holder keys test fixture construction before native proving. A public-context mismatch makes a pair ineligible; an unexpected rejection is retained as a functional result. A transcript mismatch is a candidate counterexample requiring attribution, repetition, and a check for public-input or experimental confounds.

4.3 Comparing transcripts

The decoder handles form data, JSON, compact JWTs, and encoded JSON while preserving repeated parameters and unknown siblings. Before masking cryptographic fields, it scans raw and decoded traffic for hidden fixture values. This can detect a disclosure repeated identically in every session. Differential comparison then catches unrecognized values, including credential-derived hashes that a marker scan would miss.

Fresh nonce, state, and request identifiers are renamed only after their values are checked against the session record. The renaming preserves repeated uses across messages. An opaque exemption applies to a particular scalar proof field; its decoded byte length remains visible, and its enclosing object remains comparable. The exemption assumes that the proof field provides its declared confidentiality [9].

The comparator exempts one well-formed response Date value and absolute request-object issuance time, while retaining the request object’s validity interval. It compares the request-object JWS signature by algorithm and canonical length without independently verifying that signature. Verifier identity, policy inputs, message sizes, extra fields, and response behavior remain observable. Missing boundaries, unsupported encodings, and exhausted decoding limits produce coverage failures. The JSON and HTML reports separate these from passing comparisons and local measurement data [9].

For comparisons across providers, we additionally project declared implementation details: profile and circuit identifiers, proof format and length, lookup metadata, local endpoint ports, and HTTP message sizes. The same comparison keeps extra fields and hidden-value findings. Within-provider comparisons retain proof lengths and message sizes, where a change can distinguish otherwise equivalent credentials. Both modes record their exemptions in the report, so a cross-provider result describes equality under the common claim and that projection [9].

5 Evaluation

5.1 Shared claim and provider execution

The shared-claim regression uses two adult credential variants for each provider’s transcript shape. All four clean fixture sessions have the same permitted meaning. Their two within-provider pairs and four cross-provider pairs compare equivalent. Injected versions produce findings in both comparison modes. These deterministic regressions test the comparator and integration contract; reports label their simulated acceptance and proof fields as fixture-only [9].

The implemented Java execution path uses native EPFL proofs and a session-bound synthetic envelope for the aligned OpenAC age-25 profile. The authored

OpenAC circuit has a registered build target; its native witness and Spartan key artifacts are a separate build dependency. The native campaigns below use the existing OpenAC age-and-status profile and the EPFL age-25 profile, and compare hidden-input variations within each provider [9].

Those campaigns exercise Java production controller and service code through a local HTTP adapter, with in-memory persistence and provisioned issuer/status data. EPFL fixtures supply compatible payloads with the distinct JWT headers and issuer signatures required by the wire credential and circuit witness. The measured boundary is application HTTP on the laptop; mobile UI, TLS, and live authority traffic are outside the experiment.

5.2 Disclosure and binding tests

Two signed OpenAC credentials with different hidden birth dates and status indices both satisfied the native claim and produced equivalent transcripts. We then added the compact credential to a form field and its SHA-256 digest to a header. Both presentations still passed proof verification. The comparator detected private values, the additional credential-derived identifier, and changed message sizes. An analogous EPFL campaign produced the same distinction between the clean and injected variants. Across these controls, all eight native presentations were accepted [9].

Further campaigns varied authenticated hidden holder keys and expiration data. OpenAC contributed a baseline, a same-credential repeat, and a changed-holder presentation, plus a fresh pair with different valid expiration dates. EPFL contributed a baseline, its repeat, and a changed-holder presentation. Across nine sessions, eight presentations were accepted and seven eligible pairs compared equivalent. The remaining session exceeded the public-policy freshness limit and was classified as a functional rejection. Separate native controls accepted captured proofs under their original public inputs and rejected a changed OpenAC nonce or EPFL challenge [9].

5.3 Stage measurements

Table 1 reports the measured native profiles. Presentation includes witness construction and proving; submission includes the captured verifier round trip. The same measurement interface covers both providers, while each row retains its statement and witness encoding. The aligned age-25 OpenAC transport fixture is excluded from these native measurements.

The reports place these measurements beside acceptance results, transcript differences, and the comparison policy. The companion artifact contains the shared claim, circuit source, conformance corpus, comparison regressions, and native campaign assessments, with hashes and reproduction notes [9].

Measured quantity	OpenAC	EPFL
Native profile	Age + status	Age 25
Proof bytes	315,967	16,224
Witness bytes	199,642,572	311,420–311,433
Witness encoding	Uncompressed	Compressed
Presentation (s)	39.5–42.1	14.5–15.3
Submission (s)	18.2–19.4	0.072–0.137

Table 1: Stage measurements from the native laptop campaigns. Values characterize the listed profiles and their respective encodings; submission ranges cover successful requests [9].

6 Related work and scope

Noninterference relates changes in confidential inputs to permitted observations [7]. Hyperproperties describe properties of sets of executions, including information-flow relations [3]. Declassification specifies which information may be released [10]. Metamorphic testing checks relations among executions when direct output oracles are difficult to construct; Bayati Chaleshtari et al. apply it to web-system security [1]. We apply these ideas to authenticated credential fixtures, a shared claim, and selected presentation exchanges.

OID4VP and SD-JWT supply the protocol and credential foundations; OpenAC and eid-privacy supply the proof-system implementations [13, 6, 14, 5]. Our contribution connects these through a reusable integration layer, explicit statement alignment, and transcript tests driven by the claim’s permitted release. The tests assume the declared circuit semantics and the confidentiality of exempted proof fields. Their coverage is defined by the recorded observer and normalization policy. Independent audits assess the cryptographic assumptions; the framework provides repeatable tests of the surrounding integration as providers and circuits change.

References

- [1] Nazanin Bayati Chaleshtari, Fabrizio Pastore, Arda Goknil, and Lionel C. Briand. Metamorphic testing for web system security. *CoRR*, abs/2208.09505, 2022. Available at <https://doi.org/10.48550/arXiv.2208.09505>.
- [2] CEA KRYPT. Key management for e-ID: Batch issuance and renewal. Technical report, Swiss Confederation, September 2025. Version 1.1. Available at <https://github.com/swiyu-admin-ch/community/blob/main/tech-concepts/e-id-key-management-batch-issuance-and-renewal.md>.
- [3] Michael R. Clarkson and Fred B. Schneider. Hyperproperties. *Journal of Computer Security*, 18(6):1157–1210, 2010. Available at <https://doi.org/10.3233/JCS-2009-0393>.
- [4] eid-privacy contributors. Spartan backend for Noir. <https://github.com/eid-privacy/spartan-backend>. Accessed 2026-09-21.
- [5] eid-privacy contributors. zkp-pocs: Proof of concept for privacy-preserving e-ID. <https://github.com/eid-privacy/zkp-pocs>. Experiment provenance pins commit d58bc79dd65ea6560bf199ad332d3ba3746c1c08; accessed 2026-09-21.
- [6] Daniel Fett, Kristina Yasuda, and Brian Campbell. Selective disclosure for JSON web tokens. Technical Report RFC 9901, RFC Editor, November 2025. Available at <https://www.rfc-editor.org/rfc/rfc9901.html>; accessed 2026-09-21.
- [7] Joseph A. Goguen and José Meseguer. Security policies and security models. In *1982 IEEE Symposium on Security and Privacy*, pages 11–20. IEEE Computer Society, 1982. Available at <https://doi.org/10.1109/SP.1982.10014>.
- [8] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989. Available at <https://doi.org/10.1137/0218012>.
- [9] Antonio Kambiré and Martín Ochoa. Integrating and testing zero-knowledge presentations in Swiyu: Companion artifact. Sanitized evidence bundle accompanying this working draft, 2026. Contains machine-readable assessments, source and binary provenance, normalization policy, validation records, and reproduction notes; raw private captures are excluded.
- [10] Andrei Sabelfeld and David Sands. Declassification: Dimensions and principles. *Journal of Computer Security*, 17(5):517–548, 2009. Available at <https://doi.org/10.3233/JCS-2009-0352>.
- [11] Swiss Confederation. swiyu technology stack. <https://swiyu-admin-ch.github.io/technology-stack/>. Accessed 2026-09-21.
- [12] Swiss Confederation, Federal Office of Justice. swiyu verifier. <https://github.com/swiyu-admin-ch/swiyu-verifier>. Accessed 2026-09-21.
- [13] Oliver Terbu, Torsten Lodderstedt, Kristina Yasuda, Daniel Fett, and Joseph Heenan. OpenID for verifiable presentations 1.0. Final specification, OpenID Foundation, July 2025. Available at https://openid.net/specs/openid-4-verifiable-presentations-1_0-final.html; accessed 2026-09-21.
- [14] The zkID Team at PSE. OpenAC: Open design for transparent and lightweight anonymous credentials. Technical report, Ethereum Foundation, November 2025. Available at <https://github.com/privacy-ethereum/zkID/blob/main/paper/zkID.pdf>; accessed 2026-09-21.